

Introduction to Unix shell

Part 2

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Shell Scripting Fundamentals

- Shell is more than an interactive command interpreter
- Can be used as a programming language to write scripts that automate tasks
- A shell script contains commands executed by the shell
- Shell scripts can include
 - Variables
 - Control structures (like loops and conditionals)
 - Functions
- Shell scripts typically have a `.sh` file extension

- Script opens with the shebang line: `#!/bin/bash` (or other shell interpreter)
 - Introduced by D. Ritchie in Unix version 8 (1980)
 - Indicates which interpreter should execute the script
 - Must be the very first line of the script
- Comments start with `#` and are ignored by the shell
- Commands are written line by line
- Scripts must have execute permissions to be run (`chmod +x script.sh`)

- Variables store data values that can be used and manipulated within the script
- Shell variables can have different types of data: strings, integers, arrays.
- Default variable type is a string
- Variable names should start with a letter or underscore, followed by letters, numbers, or underscores
- Variable names are case-sensitive
- Defining a variable
 - `VAR_NAME="value"`
 - No spaces are allowed around the equal sign when defining variables
- Accessing a variable
 - `$VAR_NAME`
 - `${VAR_NAME}`

- Variables are by default local to the script or function in which they are defined
- To make a variable available to child processes, it must be exported to the environment
- `export VAR_NAME` makes the variable available to child processes
- Example:
 - `MY_VAR="Hello"`
 - `export MY_VAR`
 - `bash -c 'echo $MY_VAR'`
 - This will output `Hello`
 - Without the `export` command, the child process would not have access to `MY_VAR`

- The `env` command displays or sets environment variables
- Basic usage: `env [options] [VAR=value ...] [command]`
- Common uses
 - `env`: Display all current environment variables
 - `env -i command`: Execute a command with a clean environment (no inherited variables)
 - `env VAR1=value1 VAR2=value2 command`: Execute a command with specific environment variables set
- Useful options
 - `-u <variable>`: Unset a variable before executing a command
 - `-i`: Start with an empty environment
 - `-0`: Use null character as separator in output

Variable	Description
\$PATH	Colon-separated list of directories where executable programs are located
\$HOME	User's home directory
\$USER	Current username
\$SHELL	Path to the user's login shell
\$TERM	Terminal type (e.g., <code>xterm</code> , <code>linux</code> , <code>screen</code>)
\$LANG	Language and locale settings
\$LC_ALL	Overrides all <code>LC_*</code> locale variables
\$PWD	Current working directory
\$OLDPWD	Previous working directory (used by <code>cd -</code>)

Common environment variables

Variable	Description
\$EDITOR	Default text editor
\$PAGER	Default pager program (e.g., less, more)
\$LOGNAME	User's login name
\$HOSTNAME	System hostname
\$DISPLAY	X11 display for graphical applications
\$SSH_CONNECTION	Information about SSH connection (if applicable)
\$\$	Main shell PID
\$BASHPID	Shell PID (subshell-aware)

- **String variables:** Store text values
 - `GREETING="Hello, World!"`
- **Integer variables:** Store numeric values
 - `declare -i COUNT=10`
 - `((COUNT=10))`
- **Array variables:** Store multiple values in a single variable
- Two types of arrays
 - **Indexed arrays:** Use numeric indices to access elements
 - **Associative arrays:** Use string keys to access elements

- Indexed arrays
- Declaring indexed arrays
 - `declare -a FRUITS` (FRUITS is an indexed array)
 - `FRUITS=("apple" "banana" "cherry")`
 - `FRUITS=array()` (Create an empty array)
- Adding elements to an indexed array: `FRUITS+=("date")`
- Accessing array elements:
 - `$FRUITS[0]` (first element)
 - `$FRUITS[@]` (all elements)

- Declaring associative arrays
 - `declare -A COLORS` (COLORS is an associative array)
 - `COLORS=([red]="#FF0000" [green]="#00FF00" [blue]="#0000FF")`
- Adding elements to an associative array
 - `COLORS["yellow"]="#FFFF00"`
 - `COLORS+=([black]="#000000")`
- Accessing array elements:
 - `$COLORS[red]` (element with key red),
 - `$COLORS[@]` (all elements)

- Shell supports basic arithmetic operations using the `(( ))` syntax
- Supported operations include
 - addition (+)
 - subtraction (-)
 - multiplication (*)
 - division (/)
 - modulus (%)
- Example:
 - `NUM1=10` (`NUM1` actually is a string, but bash perform type casting as needed)
 - `NUM2=5`
 - `((SUM=NUM1 + NUM2))` (`SUM` will be 15 – integer type)
 - `((PRODUCT=NUM1 * NUM2))` (`PRODUCT` will be 50)

- Incrementing and decrementing variables
 - `((COUNT++))` (increment COUNT by 1)
 - `((COUNT--))` (decrement COUNT by 1)
- Using variables in arithmetic expressions
 - `((RESULT=NUM1 * (NUM2 + 2)))` (using parentheses for grouping)
- Note: Shell performs integer arithmetic only; for floating-point calculations, consider using external programs like `bc`

- Approach to generate sequences of numbers
- Basic usage
 - `{<start>..<end>}` generates the list of numbers from `start` to `end` (included)
 - `{<start>..<end>..<step>}` list with step `step`
- Examples
 - `{1..5}` returns 1 2 3 4 5
 - `{1..10..2}` returns 1 3 5 7 9
- useful especially in iteration

- Positional parameters are used to access command-line arguments passed to the script
- Common positional parameters
 - `$0`: The name of the script
 - `$1`, `$2`, ...: The first, second, etc., command-line arguments
 - `$#`: The number of command-line arguments
 - `$*`: All command-line arguments as a single string
 - `@`: All command-line arguments as an array-like structure
- Example: If the script is run as `./script.sh arg1 arg2`, then
 - `$0` is `./script.sh`
 - `$1` is `arg1`
 - `$2` is `arg2`
 - `$#` is `2`

- The `shift` command removes positional parameters from the left
- Basic usage: `shift [n]`
- Shifts positional parameters to the left by `n` positions (default is 1)
 - `$1` becomes `$2`
 - `$2` becomes `$3`
 - And so on...
- The shifted-off parameter is discarded
- `$0` (script name) is not affected
- `$#` is decremented accordingly
- Useful for processing a variable number of arguments in loops

shift command examples

```
#!/bin/bash
#use with $0 prova1 prova2 prova3 prova4 prova5
echo "Initial arguments: $@"
echo "Number of arguments: $#"
```

shift

```
echo "After shift: $@"
echo "Number of arguments: $#"
```

shift 2

```
echo "After shift 2: $@"
echo "Number of arguments: $#"
```

- Every command executed in a shell script returns an **exit status** (aka **return code**)
- Exit status indicates the success or failure of the command
- By convention, an exit status of 0 indicates success, while any non-zero value indicates failure or an error
- The special variable `$?` holds the exit status of the last executed command
- Example
 - `ls /nonexistent_directory`
 - `echo $?` (will output a non-zero value indicating failure)
 - `ls /home`
 - `echo $?` (will output 0 indicating success)
- Scripts can use exit statuses to control flow and handle errors appropriately
- `exit <status>` command can be used to set the exit status of the script itself

- Input to a shell script can be obtained from the user using the `read` command
- Basic usage: `read [options] <variable_name>`
- Common options
 - `-p <prompt>`: Display a prompt before reading input
 - `-a <array_name>`: Read input into an array
 - `-t <timeout>`: Specify a timeout (in seconds) for input
 - `-s`: Silent mode (do not echo input, useful for passwords)
- Example:
 - `read -p "Enter your name: " USER_NAME`
 - `echo "Hello, $USER_NAME!"`
- Multiple variables can be read in a single `read` command
- When parsing the input, values of different variables are separated by spaces
 - `read -p "Enter your first and last name: " FIRST_NAME LAST_NAME`
 - `echo -e "Hello, $FIRST_NAME $LAST_NAME!"`

- When reading more than one variable the separator can be set arbitrarily
- The variable `IFS` represents the Internal Field Separator
- the default separator is the string " `\t\n`"
 - Space
 - Tab
 - Newlines

Typical alterantive values

- `IFS=""` each line is read as a whole variable (read still operates line by line)
- `IFS=$'\n'` explicitly read line by line
- `IFS=','` for example in CSV files

- Shell can collect the output of a command and store it in a variable using command substitution
- Two syntaxes for command substitution
 - Backticks: ``command`` – derived from Bourne shell
 - Dollar-parentheses: `$(command)` – derived from Korn shell
- Example
 - Using backticks: `CURRENT_DATE=`date +%Y-%m-%d``
 - Using dollar-parentheses: `CURRENT_DATE=$(date +%Y-%m-%d)`
 - Both commands store the current date in the `CURRENT_DATE` variable
- Command substitution can involve execution of complex command, possibly using pipes and redirection

Backticks ` vs. Dollar-Parentheses \$()

Feature	Backticks `	Dollar-Parentheses \$()
Readability	Poor, especially in complex scripts. Backticks can be easily confused with single quotes (apostrophes).	Excellent. The clear structure makes it easy to spot command substitutions
Nesting	Difficult or Impossible. Nested backticks require escaping the inner backticks with a backslash, which is error-prone.	Simple and Clean. Commands can be nested directly without escaping (e.g., <code>\$(command1 \$(command2))</code>)
Standard	Considered legacy or outdated. Still supported for backward compatibility.	The modern and preferred standard, introduced in the Korn shell and adopted by Bash and other modern shells

- A **subshell** is a child shell process created by the parent shell to execute commands
- The subshell inherits the environment of the parent shell but runs in a separate process
- Changes made in a subshell (variables, working directory) **do not affect** the parent shell
- Useful for isolating command execution and protecting the parent environment
- Subshells are created in several scenarios
 - Command grouping with parentheses: `(command1; command2)`
 - Piping: `command1 | command2`
 - Background execution: `command &`
 - Command substitution: `$(command)` or ``command``
- The `()` notation explicitly creates a subshell for command grouping
 - Example: `(cd /tmp; pwd)` changes to `/tmp`, but the parent shell remains in the original directory


```
#!/bin/bash
# In the main shell
echo "Main shell PID (\$\$): $$"
echo "Main shell BASHPID: $BASHPID"

# In a subshell created with parentheses
(echo "Subshell PID (\$\$): $$"
 echo "Subshell BASHPID: $BASHPID")

# In a subshell created with command substitution
SUBPID=$(echo "PID from command substitution: $$; BASHPID: $BASHPID")
echo "$SUBPID"
```

- Single Quotes (' ')
 - Preserve the literal value of each character within the quotes
 - No variable expansion or command substitution occurs
 - Example: `echo 'Hello, $USER!'` outputs `Hello, $USER!`
- Double Quotes (" ")
 - Preserve the literal value of most characters, but allow for variable expansion and command substitution
 - Example: `echo "Hello, $USER!"` outputs `Hello, <actual username>!`
- Backticks (` `)
 - Used for command substitution (as discussed earlier)
 - The output of the command within backticks is substituted into the surrounding text
 - Example: `echo "Today is `date +%A`"` outputs `Today is <current day>`

Lab experience

- Write a script that accepts a single directory path as a positional parameter.
- The script should compress (e.g., using `tar`) that directory
- The name of the archive must incorporate the current date
- The script must print a success message including the size of the newly created archive.

Useful command: tar

- The `tar` command is used to create and manipulate archive files
 - `tar=tape archive`
 - Supports various compression methods (e.g., gzip, bzip2, xz)
- Basic usage: `tar [options] <archive_name> <file_or_directory>`

- **-c**: Create a new archive
- **-x**: Extract files from an archive
- **-t**: List the contents of an archive
- **-v**: Verbosely list files processed
- **-f <archive_name>**: Specify the name of the archive file; - for stdin/stdout
- **-z**: Compress the archive using gzip
- **-j**: Compress the archive using bzip2
- **-J**: Compress the archive using xz
- **-S**: Handle sparse files efficiently
- **-C <directory>**: Change to <directory> before performing any operations
- **-k**: keep old files when extracting (**--keep-newer-files**) keep files if newer
- **--remove-files**: remove files after adding them to the archive

- Create a gzip-compressed archive of directory with name `archive.tar.gz`
 - `tar -czvf archive.tar.gz /path/to/directory`
- Extracts the contents of archive `archive.tar.xz` into the current directory
 - `tar -xJvf archive.tar.gz`
- Lists the contents of the archive `archive.tar.bz2`
 - `tar -tjvf archive.tar.bz2`

- The `date` command is used to display or set the system date and time
- Basic usage: `date [options] [+format]`
- Common options
 - `-u`: Display or set the date in Coordinated Universal Time (UTC)
 - `-R`: Output date in RFC 2822 format
 - `-d <string>`: Display the date specified by `<string>`
- Format specifiers
 - `%Y`: Year (e.g., 2024)
 - `%m`: Month (01-12)
 - `%d`: Day of the month (01-31)
 - `%H`: Hour (00-23)
 - `%M`: Minute (00-59)
 - `%S`: Second (00-59)
- Example: `date +%Y%m%d_%H%M%S` outputs the current date and time in the format `YYYYMMDD_HHMMSS`

Useful command: `basename` and `dirname`

- The `basename` command extracts the filename from a given path
 - Basic usage: `basename <path> [suffix]`
 - Example: `basename /home/user/file.txt` outputs `file.txt`
 - Example with suffix: `basename /home/user/file.txt .txt` outputs `file`
- The `dirname` command extracts the directory path from a given path
 - Basic usage: `dirname <path>`
 - Example: `dirname /home/user/file.txt` outputs `/home/user`

```
#!/bin/bash
DIR_PATH="$1"
# Removing all checks (will be added after conditional statements are covered)
# Create a timestamp for the archive name
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
ARCHIVE_NAME="$(basename "$DIR_PATH")_${TIMESTAMP}.tar.gz"
# Compress the directory
tar -czf "$ARCHIVE_NAME" -C "$(dirname "$DIR_PATH")" "$(
basename "$DIR_PATH")"
# Check the exit status of the tar command
TAR_STATUS=$?
# Check if the compression was successful
ARCHIVE_SIZE=$(du -h "$ARCHIVE_NAME" | cut -f1)
echo "Backup returned $TAR_STATUS: $ARCHIVE_NAME ($ARCHIVE_SIZE)"
```

- Write a script that prompts the user for their name and favorite color
- Stores the input in a variable,
- Prints a personalized greeting using the stored variables
- Enhancement: Print the favorite color in that color using ANSI escape sequences

```
#!/bin/bash
# Prompt the user for their name
read -p "Enter your name: " USER_NAME
# Prompt the user for their favorite color
read -p "Enter your favorite color: " FAVORITE_COLOR
# Print a personalized greeting
echo "Hello, $USER_NAME! Your favorite color is: $FAVORITE_COLOR."
```

Colorful User Profile Script

```
#!/bin/bash
COLOR_RESET="\e[0m"
declare -A ANSI_COLORS
ANSI_COLORS=(["black"]="\e[30m" ["red"]="\e[31m" ["green"]="\e[32m"
["yellow"]="\e[33m" ["blue"]="\e[34m" ["magenta"]="\e[35m"
["cyan"]="\e[36m" ["white"]="\e[37m")
# Prompt the user for their name
read -p "Enter your name: " USER_NAME
# Prompt the user for their favorite color
read -p "Enter your favorite color: " FAVORITE_COLOR
# Print a personalized greeting
# NOTE: using ${FAVORITE_COLOR,,} to convert to lowercase
echo -ne "Hello, $USER_NAME! Your favorite color is: "
echo -e "${ANSI_COLORS[${FAVORITE_COLOR,,}]}$FAVORITE_COLOR${COLOR_RESET}."
```

- Create a directory for deploying an application.
 - Rely on three variables named `BASE` (e.g. `/home/user/prjs`), `PROJECT` (e.g. `app`), and `VERSION` (e.g. `1.0.2`)
 - Create a directory layout for the deployment starting with `/home/user/prjs/app/1.0.4` and with the directories `src`, `build`, `docs`
- Shopping list
 - Create an array `GROCERIES` containing `bread`, `milk`, `eggs`, `coffee`
 - Print the total number of items in the list
 - Print a reminder that indicates the first element in the list
- Math galore
 - Given two variables `BIRTH_YEAR`, `CURRENT_YEAR` and a third variable `MONTHLY_BUDGET`
 - Compute the approximate age and the daily budget (assuming 30-days months)

Bonus tips and tricks

- The `.profile` file is a shell script that is executed when a user logs in to a Unix-like operating system
 - It is typically located in the user's home directory (`~/.profile`)
 - It can read from the system-wide configuration file `/etc/profile`
 - It is used to set up the user's environment and customize the shell behavior
- Purpose of `.profile` file
 - Set environment variables (e.g., `PATH`, `EDITOR`)
 - Configure shell options and behaviors
 - Define aliases and functions for frequently used commands
 - Customize the shell prompt (using `PS1` variable)
- Different shells may use different configuration files (e.g., `.bashrc`, `.bash_profile`, etc.)
 - `.bashrc` is executed for non-login interactive shells
 - `.bash_profile` is executed for login shells.
 - It usually sources `.bashrc` to ensure consistent behavior

- The `PS1` variable defines the primary prompt string displayed in the shell
- It can be customized to include various information, such as the username, hostname, current working directory, and more
- Common escape sequences used in `PS1`
 - `\u`: Username of the current user
 - `\h`: Hostname of the machine
 - `\w`: Current working directory
 - `\$`: Displays `$` for regular users and `#` for the root user
 - `\t`: Current time in 24-hour HH:MM:SS format
 - `\n`: New line
- Example: Setting a custom prompt
 - `export PS1="\u@\h:\w\$ "`
 - This will display the prompt as `username@hostname:current_directory$`

- The `bc` command is an arbitrary precision calculator language and interpreter
- Useful for performing floating-point arithmetic and mathematical operations in shell scripts
- Basic usage: `bc [options] [file]`
- Common options
 - `-l`: Load the standard math library (provides functions like `sin`, `cos`, `sqrt`, etc.)
 - `-w`: Warn about non-standard extensions
 - `-s`: Process only the syntax (do not execute)
- Input can be provided via files or piped from other commands (e.g., `echo`)

- bc supports standard arithmetic operators
 - `+`, `-`, `*`, `/`: Basic arithmetic
 - `%`: Modulus (remainder)
 - `^`: Exponentiation
- Variables can be defined and used
 - `x = 10` (assignment)
 - `y = x + 5` (using variables in expressions)
- The `scale` variable controls decimal precision
 - `scale = 2` (sets 2 decimal places)
 - `10 / 3` (without `scale`: result is 3; with `scale=2`: result is 3.33)
- Results are printed automatically or using `print`

- When using `bc -l`, the standard math library provides functions
 - `sqrt(x)`: Square root of x
 - `s(x)` – sin, `c(x)` – cos, `a(x)` – arctan: Trigonometric functions (angle in radians)
 - `l(x)`: Natural logarithm
 - `e(x)`: Exponential function
 - `j(n, x)`: Bessel J_n function
 - `scale(x)`: Get or set the scale for x

```
# Simple arithmetic
echo "10 + 5" | bc # Output: 15
# Floating-point division
echo "scale=2; 10 / 3" | bc # Output: 3.33
# Exponentiation
echo "2 ^ 10" | bc # Output: 1024
# Using variables
echo "x = 50; y = 30; x * y" | bc # Output: 1500
# Mathematical functions (requires -l option)
echo "scale=4; sqrt(16)" | bc -l # Output: 4.0000
# Calculate pi
echo "scale=10; 4 * a(1)" | bc -l # Output: 3.1415926535
```

```
#!/bin/bash
# Calculate average of three numbers
NUM1=85
NUM2=90
NUM3=78
AVERAGE=$(echo "scale=2; ($NUM1 + $NUM2 + $NUM3) / 3" | bc)
echo "Average: $AVERAGE"
```

```
# Calculate percentage
FS="/tmp"
TOTAL=$(df $FS --output=size | tail -n 1)
USED=$(df $FS --output=used | tail -n 1)
PERCENTAGE=$(echo "scale=1; ($USED / $TOTAL) * 100" | bc)
echo "Disk Usage: (${USED}/${TOTAL}) $PERCENTAGE%"
```

```
# Using bc with variables from user input
read -p "Enter a number: " NUM
SQUARED=$(echo "$NUM ^ 2" | bc)
echo "Square of $NUM is $SQUARED"
```

- **oh-my-bash** is an open-source framework for managing your Bash configuration
- It provides a collection of themes, plugins, and useful functions to enhance the Bash shell experience
- Features of **oh-my-bash**
 - Easy installation and setup
 - A wide variety of themes to customize the appearance of the shell prompt
 - Numerous plugins for added functionality (e.g., git integration, syntax highlighting, auto-suggestions)
 - Regular updates and a large community of users
- Installation: Follow the instructions on the **oh-my-bash** GitHub repository
- After installation, you can customize your shell by editing the `.bashrc` file

Pimp my shell with oh-my-bash

```
[11:27 PM]-[laguan@israel]-[~/.../Tools/oh-my-bash]- [themes-screenshoots ?:1 X]
$ git status
On branch themes-screenshoots
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    themes/THEMES.md

nothing added to commit but untracked files present (use "git add" to track)

[11:27 PM]-[laguan@israel]-[~/.../Tools/oh-my-bash]- [themes-screenshoots ?:1 X]
$ ls -la
total 96
drwxr-xr-x 14 laguan 4096 Dec  2 23:17 .
drwxr-xr-x  4 laguan 4096 Dec  2 23:16 ..
drwxr-xr-x  2 laguan 4096 Dec  2 23:17 aliases
```

```
laguan@israel 23:46:57
~/Projects/Tools
> cd oh-my-bash/
/home/laguan/Projects/Tools/oh-my-bash
laguan@israel 23:47:00
~/.../Tools/oh-my-bash ± (themes-screenshoots ?:9 X)
> git status
On branch themes-screenshoots
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    themes/90210/90210-dark.png
    themes/THEMES.md
    themes/agnoster/agnoster-dark.png
    themes/axin/axin-dark.png
    themes/hakke/hakke-dark.png
```

```
israel @ ~/Projects/Tools ruby-2.5.5
> _ cd oh-my-bash/
/home/laguan/Projects/Tools/oh-my-bash

israel @ ~/.../Tools/oh-my-bash ruby-2.5.5 429ffc1 themes-screenshoots ?:12 X
> _ git status
On branch themes-screenshoots
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    themes/90210/90210-dark.png
    themes/THEMES.md
    themes/agnoster/agnoster-dark.png
    themes/axin/axin-dark.png
    themes/hakke/hakke-dark.png
```

```
00 israel in ~/.../Tools/oh-my-bash
± |themes-screenshoots ?:21 X| → cd ...
..

00 israel in ~/Projects/Tools
o → cd oh-my-bash/
/home/laguan/Projects/Tools/oh-my-bash

00 israel in ~/.../Tools/oh-my-bash
± |themes-screenshoots ?:21 X| → git status
On branch themes-screenshoots
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    themes/90210/90210-dark.png
    themes/THEMES.md
```